

# Gradient estimates and the training clock

Choose how much data measures each step, then name the unit in which training progress is counted.

## THE QUESTION

How much data should measure each gradient step, and how far should the parameters move?

## THE INTUITION

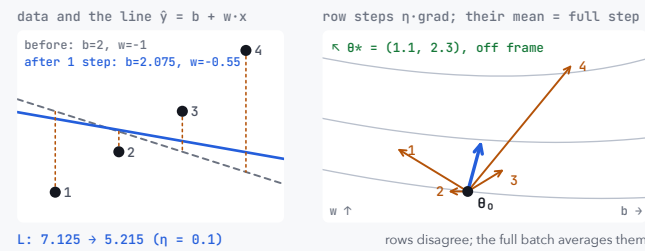
Backprop measures a slope; optimization decides which examples measure it. A random minibatch gives a noisy but unbiased estimate of the full gradient, so many cheap steps can beat one exact one.

## KEY TAKEAWAY

A sampled gradient is right on average, not on every step. One scalar  $\eta$  must serve every direction, so a ravine forces a speed-stability trade-off.

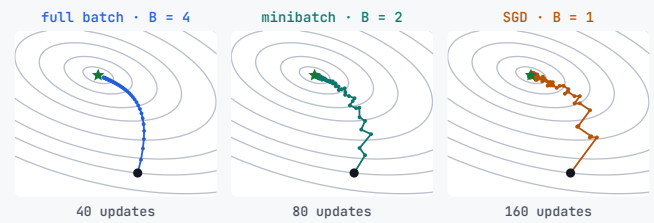
sample a minibatch → loss → backward →  $\hat{g}$  → scale by  $\eta$  →  $\theta \leftarrow \theta - \eta \hat{g}$  → next batch

### 1 Objective and vanilla update



For  $N$  examples,  $\ell_i(\theta) = \ell(f_\theta(x_i), y_i)$  and  $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_i(\theta)$ . Full-batch GD:  $g_t = \nabla \mathcal{L}(\theta_t)$ ,  $\theta_{t+1} = \theta_t - \eta g_t$ , with learning rate  $\eta > 0$ . The gradient points uphill; subtracting it moves locally downhill. Above: 4 rows,  $\ell_i = \frac{1}{2}(b + wx_i - y_i)^2$ .

### 2 Full batch, SGD, minibatch



| method     | $B$         | updates / epoch     | noise                            |
|------------|-------------|---------------------|----------------------------------|
| full batch | $N$         | 1                   | none: exact $\nabla \mathcal{L}$ |
| SGD        | 1           | $N$                 | highest                          |
| minibatch  | $1 < B < N$ | $\lceil N/B \rceil$ | intermediate                     |

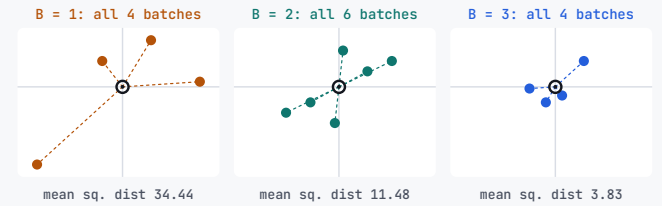
Minibatch estimator:  $\hat{g}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t)$ . Above: 40 epochs at  $\eta = 0.1$ , one shuffled run each. "SGD" in libraries usually means this whole family; the dataloader's batch size tells you which member you ran.

### 3 Name the clock

|         |                    |                     |                         |
|---------|--------------------|---------------------|-------------------------|
| $B = 4$ | <div>4 2 3 1</div> | 1 update per epoch  | $\lceil 4/4 \rceil = 1$ |
| $B = 2$ | <div>4 2 3 1</div> | 2 updates per epoch | $\lceil 4/2 \rceil = 2$ |
| $B = 1$ | <div>4 2 3 1</div> | 4 updates per epoch | $\lceil 4/1 \rceil = 4$ |

$N$  examples in batches of  $B$  give  $K = \lceil N/B \rceil$  iterations per epoch. **Epoch**: one pass through the data. **Iteration**: one minibatch loop body. **Update**: one executed parameter change. Usually iteration = update; gradient accumulation and skipped optimizer steps are the exceptions.

### 4 Why a sampled gradient can work

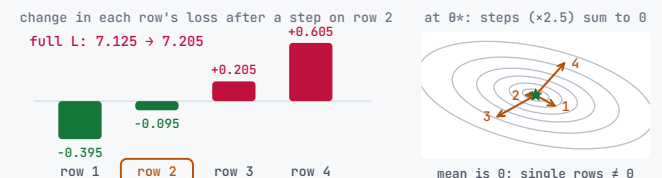


Draw  $I_t$  uniformly from  $\{1, \dots, N\}$  and use  $\hat{g}_t = \nabla \ell_{I_t}(\theta_t)$ . Holding  $\theta_t$  fixed,

$$\mathbb{E}[\hat{g}_t | \theta_t] = \frac{1}{N} \sum_i \nabla \ell_i(\theta_t) = \nabla \mathcal{L}(\theta_t).$$

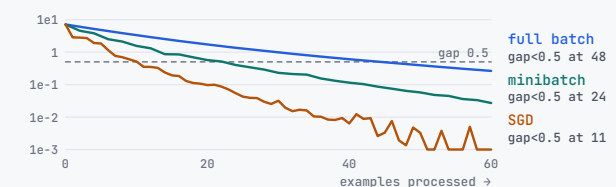
The estimator is **unbiased**: correct at its centre across repeated draws. Larger batches keep the centre and shrink the spread.

### 5 What unbiased does not mean



One sampled gradient need not be close to the full gradient, and one valid SGD step need not reduce the full loss. It only satisfies  $\mathbb{E}[\Delta \theta_t | \theta_t] = -\eta \nabla \mathcal{L}(\theta_t)$ . Even at  $\theta^*$  single rows keep pushing, so constant-rate SGD never fully settles.

### 6 Batch size: a systems-and-statistics knob



**Smaller  $B$**   
more updates per epoch  
noisier estimates  
smaller parallel workload  
may regularize implicitly

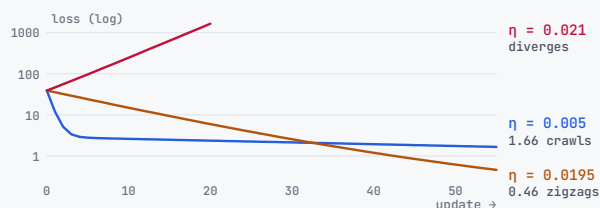
**Larger  $B$**   
fewer updates per epoch  
less estimator spread  
larger parallel workload  
may need rate retuning

Compare runs by examples processed or updates, not only epochs.

# Step size, geometry, and diagnosis

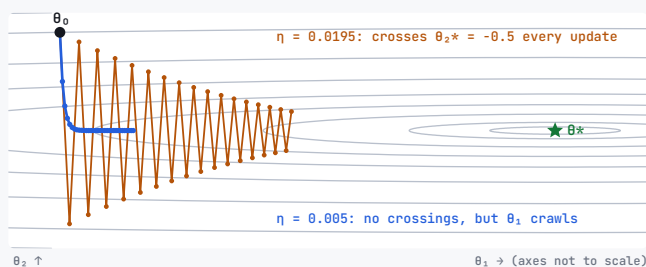
An exact gradient can still make poor progress when one scalar learning rate must serve very different directions.

## 7 The learning rate scales every coordinate



$\Delta\theta_t = -\eta\hat{g}_t$ . Too small: safe but slow. Too large: overshoot, oscillation or divergence. A useful rate makes early progress without destroying stability. **Diagnostic:** plot training loss against optimizer updates and log the rate actually used at each update.

## 8 Ravines create a speed-stability trade-off



$\mathcal{L}(\theta) = \frac{1}{2}(\theta_1 - 1)^2 + 50(\theta_2 + 0.5)^2$ ,  $\nabla\mathcal{L} = (\theta_1 - 1, 100(\theta_2 + 0.5))$ . One direction is  $100\times$  steeper. A rate large enough to move along  $\theta_1$  crosses the narrow valley in  $\theta_2$  again and again; a stable small rate crawls along the floor.

## 9 Concrete first step

At  $\theta_0 = (-1.4, 0.35)$ ,  $g_0 = (-2.4, 85)$ . With  $\eta = 0.0195$ :  $\Delta\theta_0 = (0.0468, -1.6575)$ ,  $\theta_1 = (-1.3532, -1.3075)$ . Downhill locally, yet it crosses  $\theta_2^* = -0.5$ . Each GD step multiplies the errors by  $1 - \eta(\theta_1)$  and  $1 - 100\eta(\theta_2)$ :

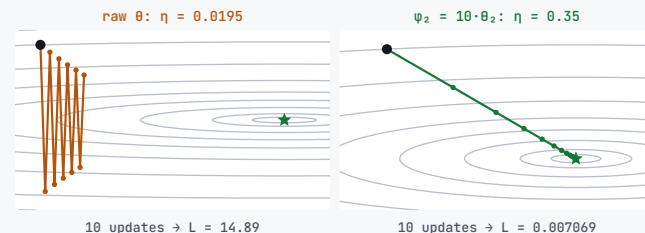
| $\eta$ | $\theta_1$ factor | $\theta_2$ factor | $\theta_2$ error |
|--------|-------------------|-------------------|------------------|
| 0.005  | 0.995             | 0.5               | halves           |
| 0.0195 | 0.9805            | -0.95             | flips, shrinks   |
| 0.021  | 0.979             | -1.1              | flips, grows     |

## 10 Pause and think

**Why can one SGD step raise the loss?** It follows one row's opinion; only the average over rows points downhill.

**Why not simply shrink  $\eta$ ?**  $\theta_2$  then settles, but  $\theta_1$  keeps 99.5% of its error per step at  $\eta = 0.005$ .

## 11 Fix geometry before blaming the optimizer



Standardizing  $x_2$  (values  $\pm 10 \rightarrow \pm 1$ ) is the substitution  $\varphi_2 = 10\theta_2$ : both curvatures become 1, so a  $18\times$  larger rate is stable. Same starting predictor, 10 updates each.

- ✓ standardize features
- ✓ inspect contours or per-parameter update scales
- ✓ check exploding gradients, invalid targets, reduction factors
- ✓ retune  $\eta$  after changing batch size or normalization

## 12 A disciplined batch-size comparison

Hold the model, data order, loss reduction and total example budget fixed. For each  $B$  log: (a) examples processed, (b) optimizer updates, (c) wall-clock time and device utilization, (d) training and validation metrics, (e) gradient or update norms. If  $B$  changes, do not assume the old learning rate is still optimal.

## 13 Minimal PyTorch loop

```
for xb, yb in loader:
    opt.zero_grad(set_to_none=True)
    loss = loss_fn(model(xb), yb)
    loss.backward() # batch gradient
    opt.step()      # parameter update
```

PyTorch accumulates gradients, so clear them every iteration. Record the loss reduction (mean vs sum): it rescales the gradient and hence the effective rate.

## 14 Symptom $\rightarrow$ first check

| symptom       | first check                               |
|---------------|-------------------------------------------|
| loss explodes | $\eta$ , invalid values, reduction scale  |
| loss zigzags  | feature scale / curvature; reduce $\eta$  |
| loss flat     | zeroed gradients, dead units, tiny $\eta$ |
| very noisy    | shuffling, batch size, outliers, logging  |
| run is slow   | examples/s and updates/s                  |

**Report batch size, loss reduction, learning rate, update count and example budget; otherwise the experiment is underspecified.**

Numbers from the Lecture 5 running examples (4-row line fit; the  $\theta_1/\theta_2$  ravine) recomputed in `briefs/L05-data.py`; the shuffled runs use seed 5 and are one run each, not a ranking. Slides and lesson: [nipunbatra.github.io/dl-2026](https://nipunbatra.github.io/dl-2026).